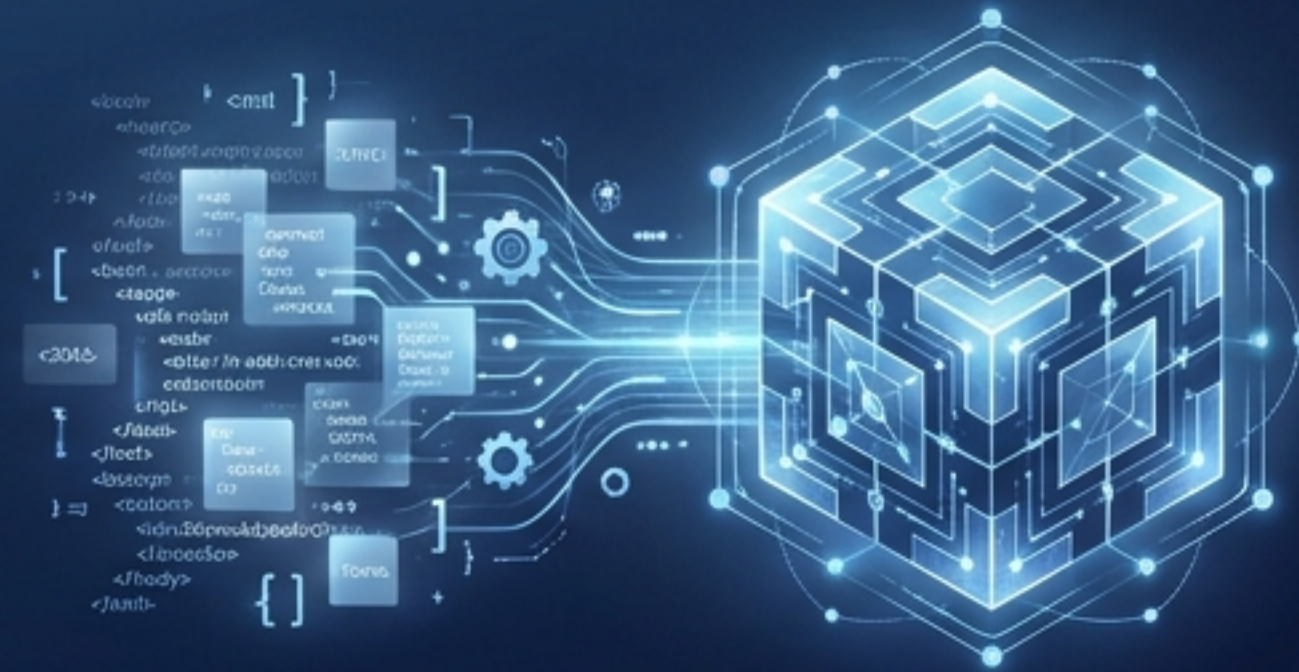


Tema 13. Lenguaje de marcas y sistemas de gestión de información.

Transformaciones XSLT

De datos crudos a información
estructurada mediante
programación declarativa.

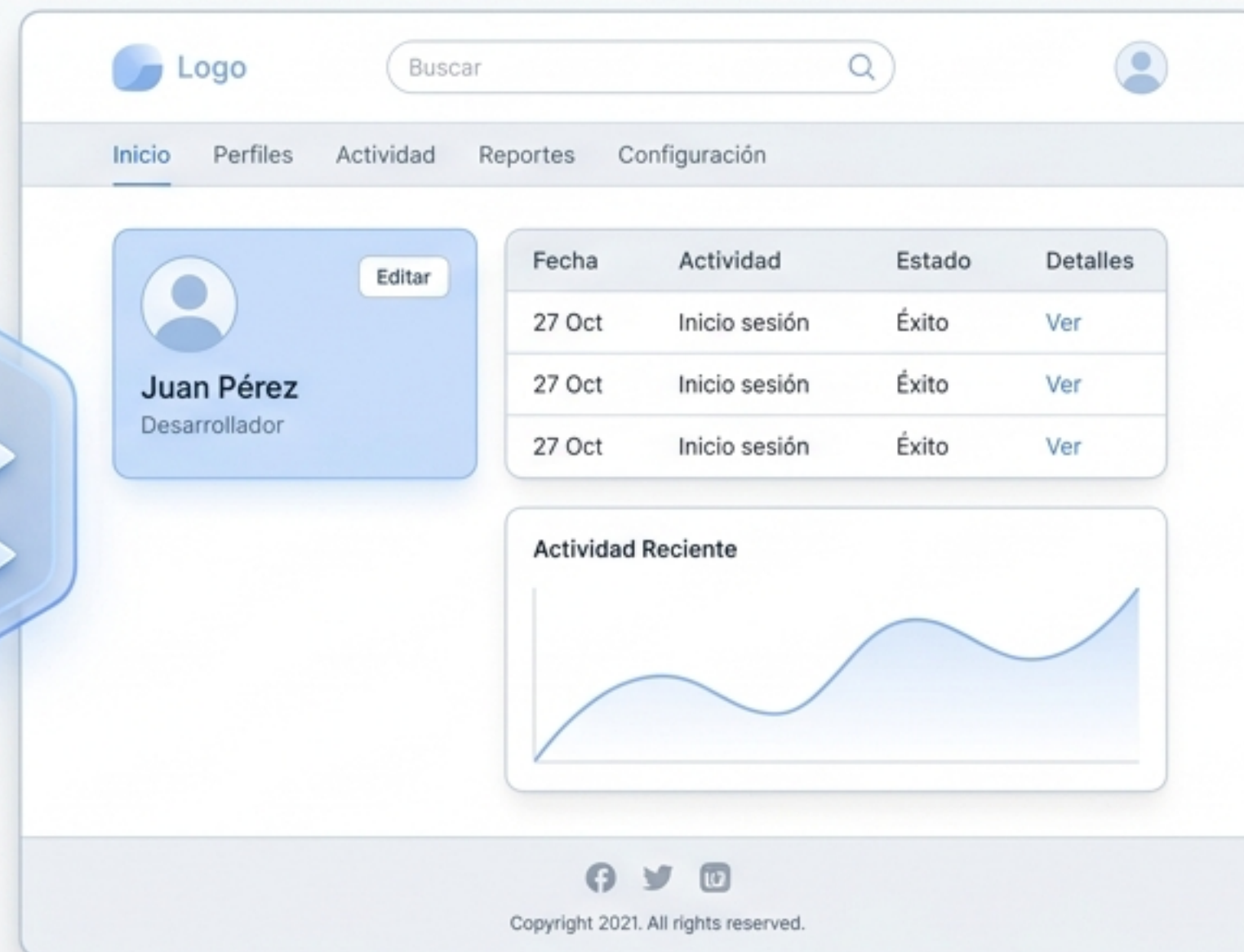


El Origen: Datos Crudos

```
<database>
  <record id="A101">
    <name>Juan Pérez</name>
    <age>34</age>
    <data type="profile">...</data>
  </record>
  <record id="B202">
    <name>Ana Gómez</name>
    <age>29</age>
    <log entry="2023-10-27">Login successful</log>
    <data type="activity">...</data>
  </record>
  <configuration>
    <setting key="max_users">500</setting>
    <setting key="timeout">600</setting>
  </configuration>
  <report id="R99" status="pending">Monthly Summary</report>
  <data>
    encryP25cHn5VhhANy68i9Xq6hKis1/Rj1kGNsKQc0bQRQ2E0yRfKpUI
    <Rvh5kkb3kWjkkYxGYmYgsK312sTYPa2IpKcR3VgTEq8899EWv1l'wFY>
      <node encryptim{h189b2522?node}..</node>
      <node encryptim{âb8734312?node}..</node>
    </data>
  </record>
</database>
```



El Destino: Interfaz Humana



XML estructura la información perfectamente para los sistemas, pero ignora al usuario final.

- **El Problema:** Frecuentemente recibimos ficheros XML de sistemas legados.
- **La Limitación:** Su formato en árbol es excelente para almacenamiento, pero incomprensible en pantalla.
- **El Reto:** ¿Cómo convertimos datos crudos en interfaces legibles sin alterar el origen?

La solución requiere transmutación, no solo maquillaje visual.



CSS (Hojas de Estilo)

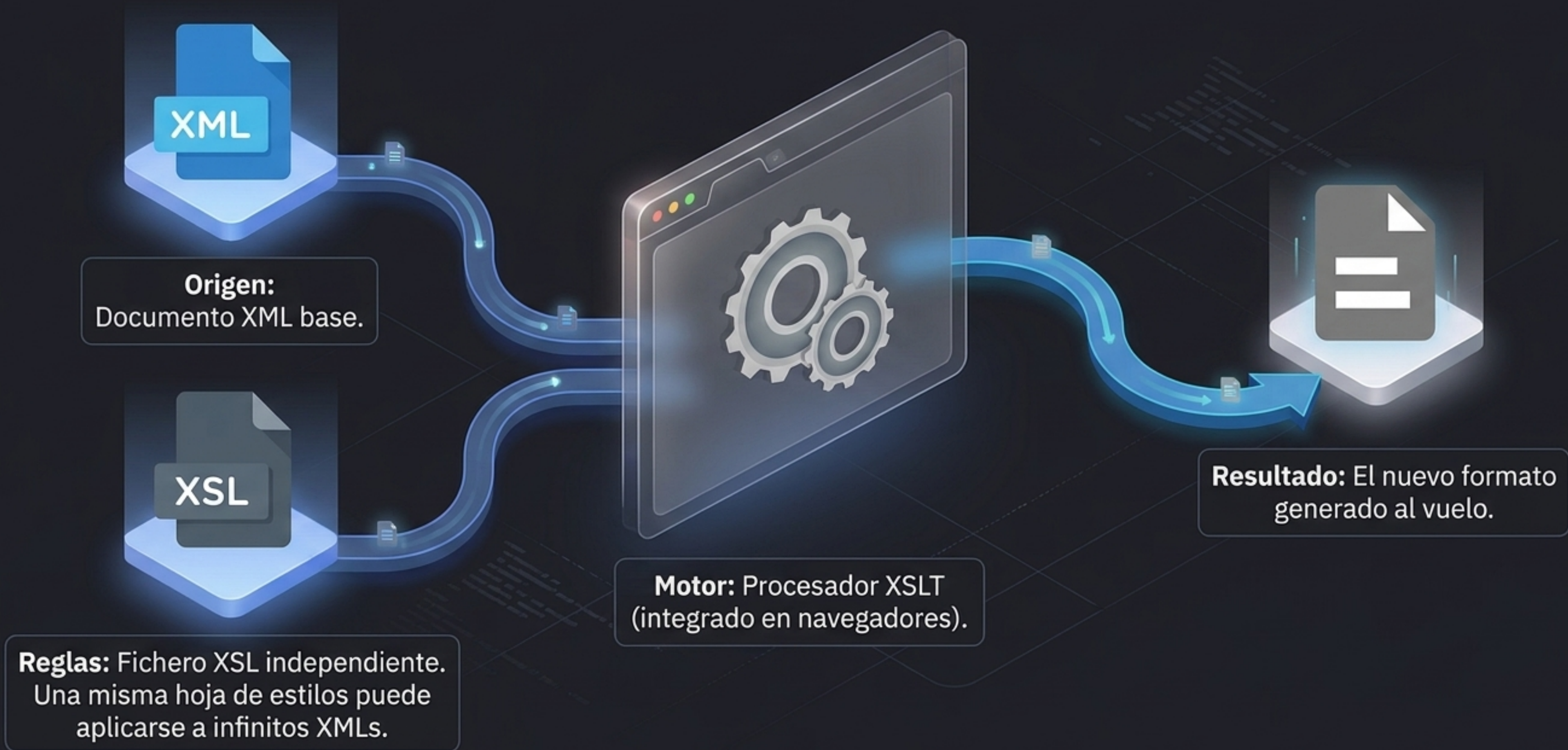
- **Función:** Maquilla el documento.
- **Efecto:** El documento original sigue siendo estructuralmente el mismo. Solo cambia su apariencia superficial (colores, tipografías, bordes).



XSLT (Transformaciones)

- **Función:** Transmuta el documento.
- **Efecto:** Lenguaje de programación declarativo. Extrae la información del XML origen y genera un documento completamente nuevo (HTML, Texto, Texto, u otro XML).

El motor de transformación opera procesando reglas independientes.



Un documento XSLT es, estructuralmente, un documento XML válido.

1. Declaración obligatoria: Al ser un derivado de XML, requiere su propia declaración inicial.

2. Elemento Raíz: Encapsula toda la lógica de transformación.

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
  ...
</xsl:stylesheet>
```

3. Espacio de Nombres (Namespace): Vital. Actúa como diccionario para que el procesador reconozca que las instrucciones con el prefijo xsl: son comandos ejecutables.

El puente de enlace se establece directamente desde el archivo origen.

Fichero XML origen

```
<?xml version="1.0"?>  
  
<?xml-stylesheet type="text/xsl"  
href="hoja_estilo.xsl"?>  
  
<datos>...</datos>
```

Fichero XSLT

Hoja de reglas esperando ser invocada.

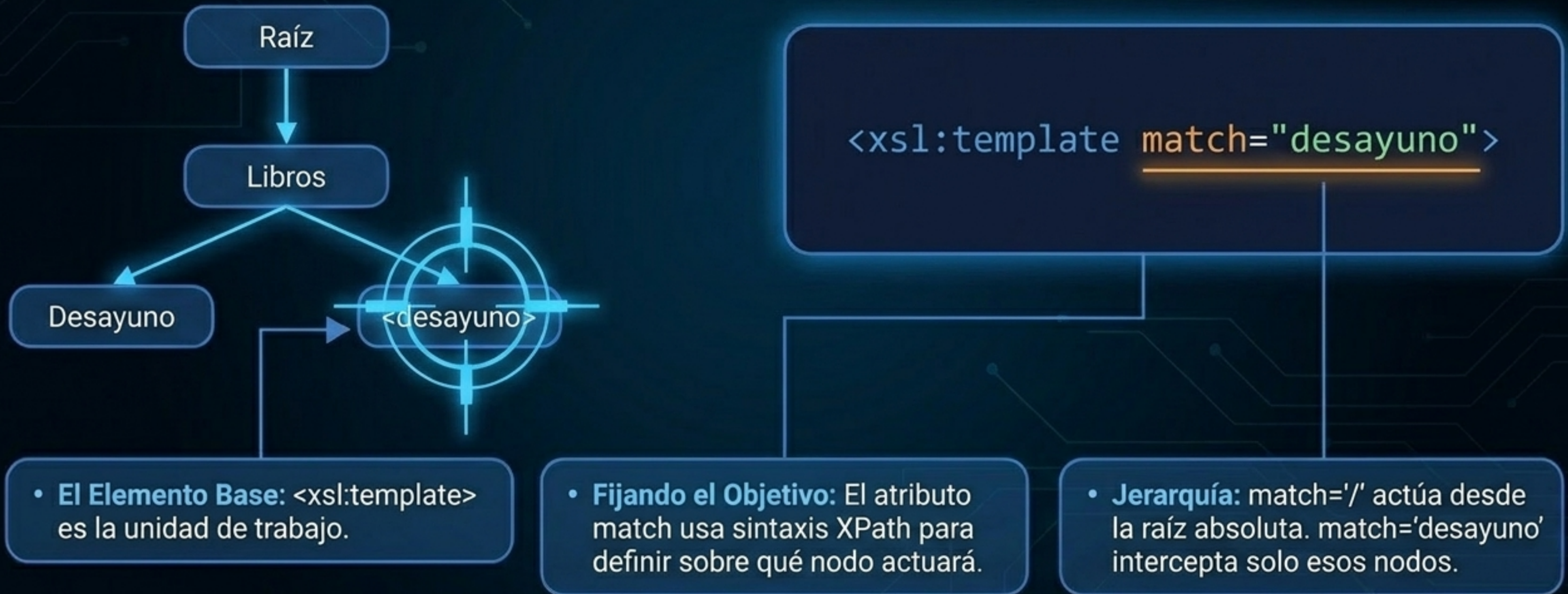
- **Instrucción de procesamiento:** Se inserta en la cabecera del XML.

- **Atributo type:** Define que el motor use el estándar (text/xsl).

- **Atributo href:** La ruta hacia el archivo de lógica.

El lenguaje opera aplicando 'Plantillas' sobre objetivos específicos.

XSLT es un lenguaje declarativo. Declaramos qué debe ocurrir cuando el motor encuentre un nodo específico, sin detallar el proceso paso a paso.



La caja de herramientas del desarrollador XSLT

1. Configuración



Elemento: `<xsl:output>`

Función: Define el ADN del archivo resultante (HTML, XML, o Texto plano).

2. Extracción



Elemento: `<xsl:value-of>`

Función: Captura el contenido crudo o los atributos exactos desde el árbol XML.

3. Inyección



Elemento: `<xsl:text>` / `<xsl:comment>`

Función: Fuerza la inserción de espacios, caracteres especiales o comentarios invisibles para depuración.

4. Control de Flujo



Elemento: `<xsl:apply-templates>`

Función: Invoca sub-rutinas y delega el procesamiento a otras plantillas específicas.

Definiendo la naturaleza del archivo resultante

```
<xsl:output method="html"  
omit-xml-declaration="yes"  
indent="yes"/>
```

[xml ☒ html text]

El conmutador principal que decide la estructura de salida.



Esencial para HTML limpio; elimina la cabecera `<?xml?>` para evitar errores en navegadores.



Auto-formatea el código resultante con tabulaciones para facilitar la lectura humana.

Nota: Este elemento de nivel superior debe colocarse inmediatamente debajo de la raíz **<stylesheet>**.

Recolección de precisión milimétrica mediante selectores

Datos crudos

```
<precio>6.95</precio>
```



Instrucción de extracción

```
<xsl:value-of select="desayuno[3]/precio"/>
```



6.95

Acción: Extrae el texto exacto del tercer desayuno.

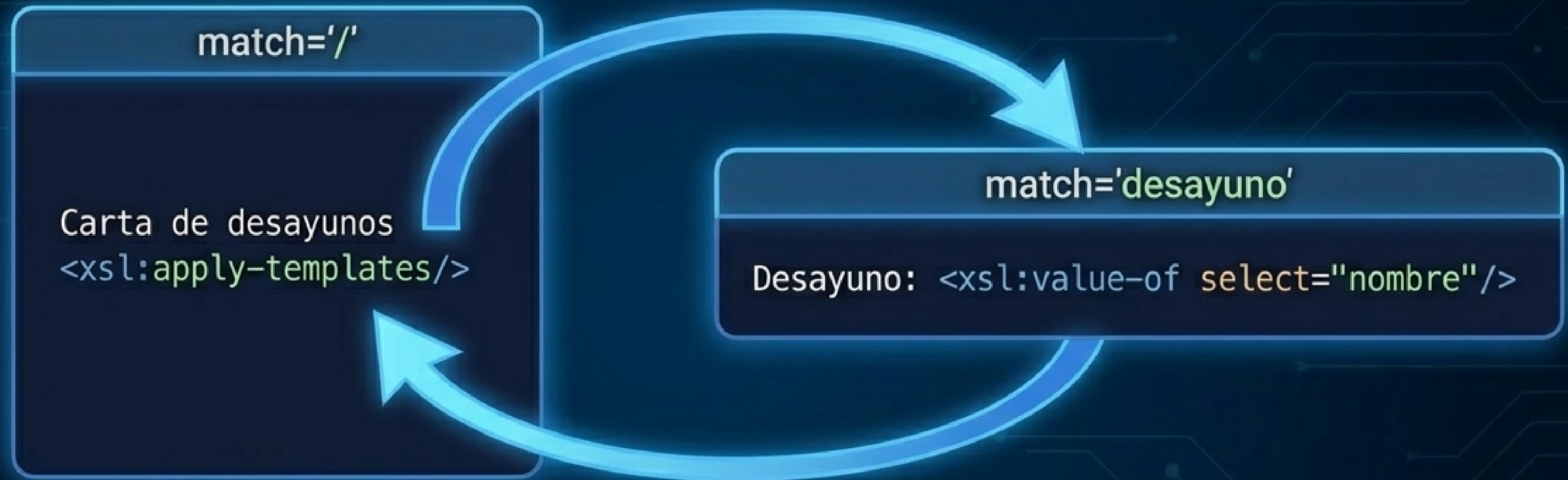
Uso de Funciones y Atributos

```
<xsl:value-of select="count(desayuno)"/>  
<xsl:value-of select="@tipo"/>
```

El atributo **select** admite funciones XPath avanzadas. **count()** suma el número total de nodos, y el prefijo **@** captura el valor de un atributo en lugar de la etiqueta interior.

Delegación de control: El salto a las sub-plantillas

Para evitar archivos caóticos, XSLT permite fragmentar el código invocando lógicas secundarias.



Sin atributo select

- ✓ Por defecto, el procesador salta e intenta procesar todos los nodos secundarios del nodo actual.

Con atributo select

- ✓ Ej: `<xsl:apply-templates select='expresion'/'>`. Fuerza el salto exclusivamente hacia los nodos que cumplan la condición indicada.

Inyectando espacios en blanco y notas de depuración

Texto Literal

```
<xsl:text disable-output-  
escaping="yes">&lt;  
</xsl:text>
```

- Añade espacios o caracteres problemáticos para el editor.
- **disable-output-escaping**: Fuerza al procesador a mostrar caracteres en bruto (como < o &) en lugar de sus entidades seguras (<).

Comentarios de Código

```
<xsl:comment>  
  Datos procesados  
</xsl:comment>
```

- Inyecta la estructura clásica de comentarios (<!-- -->) directamente en el documento generado final.
- Ideal para depurar la transformación sin ensuciar la interfaz gráfica del usuario.

La fusión de mundos: Inyección de estructuras HTML en línea

La verdadera potencia de XSLT radica en envolver los datos extraídos con código de maquetación HTML y reglas de diseño CSS.

Estructura Base:
XSLT inicializa la plantilla.

```
<xsl:template match="desayuno">
  <div style="border: solid 3px chocolate;">
    <b><xsl:value-of select="nombre"/></b><br/>
  </div>
</xsl:template>
```

Cápsulas de Diseño:
Utilizamos etiquetas HTML estáticas (div, b) con estilos CSS aplicados en línea.

Lógica Viva: La instrucción value-of inyecta el dato exacto de la base de datos dentro del marco diseñado.

Síntesis Práctica: Filtrado y formateo de calificaciones

Objetivo de Negocio: Mostrar en una web únicamente los alumnos aprobados (nota ≥ 5) del módulo 'LLMM'.

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  version="2.0">
  <xsl:output method="html"/>
  <xsl:template match="/">
    <html>
      <head><title>Alumnos Aprobados LLMM</title></head>
      <body>
        <h1>Lista de Alumnos Aprobados en LLMM</h1>
        <xsl:apply-templates
          select="//modulu[@nombre='LLMM']"/>
      </body>
    </html>
  </xsl:template>
  <xsl:template match="modulo[@nombre='LLMM']">
    <ul>
      <xsl:apply-templates select="alumno[nota>=5]"/>
    </ul>
  </xsl:template>
  <xsl:template match="alumno[nota>=5]">
    <li>
      <xsl:value-of select="nombre"/> (Nota: <xsl:value-of select="nota"/>)
    </li>
  </xsl:template>
  <xsl:template match="alumno[nota<5]">
    <!-- Los suspensos son ignorados -->
  </xsl:template>
</xsl:stylesheet>
```

1

1. Arranque: Crea el esqueleto base web (etiquetas HTML, Head, Body).

2

2. Filtrado de Módulo: Ignora el resto del instituto y delega el flujo solo al módulo deseado.

3

3. Lógica Condicional: Actúa como filtro extractivo. El nombre del estudiante solo se inyecta si su nota supera la condición. Los suspensos son ignorados.

El resultado final: De la base de datos a la pantalla del usuario



- ✓ **Integridad garantizada:** Los datos crudos han sido filtrados sin alterar un solo byte del XML original.
- ✓ **Automatización:** Si las notas cambian en la base de datos, la web se actualiza instantáneamente en la próxima recarga.
- ✓ **Separación de capas:** La lógica de extracción (XSLT) convive armónicamente con la capa de presentación visual (HTML/CSS).

XSLT es el puente definitivo entre el almacenamiento de datos estructural y la experiencia del usuario final.